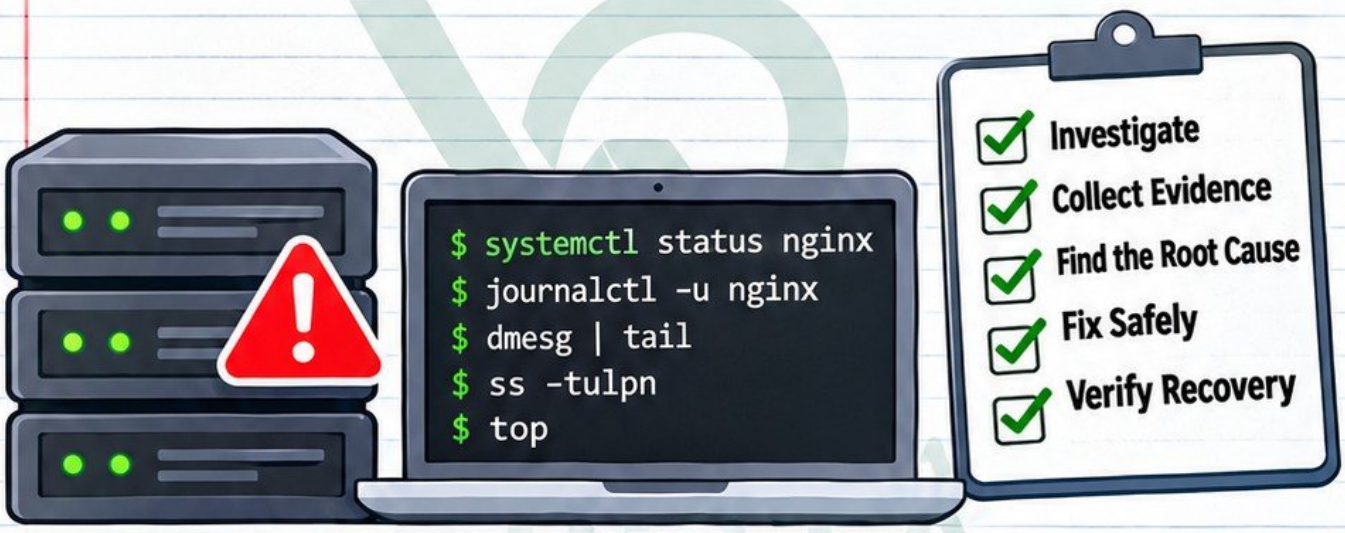
VERIQTA

How to Debug a Broken Server Like a Detective

Real-World Troubleshooting for Linux, Nginx, Cloud and Production Systems



Linux Troubleshooting | Nginx Diagnostics | Database Issues | Cloud & Networking | Container & Deployment | Monitoring & Recovery

30 Real-World Cases, Diagnoses and Fixes

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

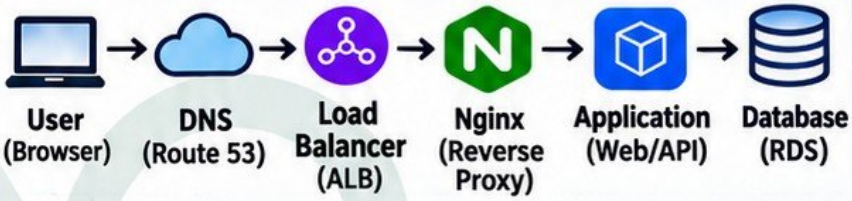
The Server Is Up, but the Business Is Down

Prove the failure using an end-to-end request path before touching the server.

The Scenario

- Users report the application is not working.
- The server shows it is up (SSH, ping, systemctl are OK).
- Business operations are still affected (e.g., no orders, no logins, no payments).

End-to-End Request Path



Investigation Steps

1. Reproduce the issue (from a user view).
2. Test each layer in the request path (DNS → LB → Nginx → App → DB).
3. Collect evidence (status, logs, metrics, and response codes).
4. Identify where the request breaks or slows down.

Key Commands and Expected Evidence

1. `curl -I https://app.example.com`
   Expected: HTTP status (e.g., 200, 502, or 504)
2. `dig app.example.com`
   Expected: IP address (DNS is resolving)
3. `curl -I http://<alb-dns>`
   Expected: LB response (e.g., 200 or 503)
4. `curl -I http://<nginx-ip>`
   Expected: Nginx response (e.g., 200, 404, or 502)

Senior Insight

- A healthy server does not mean a healthy business.
- Always validate the full request journey before changing anything on the server.

Resolution and Verification

- Fix the failure at the confirmed layer (e.g., DNS, Load Balancer, Nginx, App, or DB).
- Re-test the full request path end-to-end.
- Verify business functions (e.g., login, checkout, API calls) are working for real users.

VERIQTA

The Reboot That Destroyed the Evidence

Capture volatile process, socket, kernel, and memory evidence before recovery.

The Scenario

- A production server is slow and unresponsive.
- A restart might fix it, but it will wipe critical evidence.
- You need to capture a live snapshot first for root cause analysis.

What Gets Lost After a Reboot?

- Running Processes (PIDs, state)
- Active Network Connections (Sockets)
- Kernel State (modules, locks, queues)
- Memory Evidence (including possible leaks)

Evidence to Capture (in Order)

1. System overview and load.
2. Running processes and resource usage.
4. Open files and active network sockets.
5. Memory snapshot for analysis.
5. Kernel and system logs.

Optional (Deep Evidence)

- **lsof -nP** → Open files and handles.
- **free -h** → Memory usage.
- **vmstat 1 5** → System performance (CPU, I/O).
- **sudo gcore <PID>** → Capture a process memory dump (if needed).

Key Commands (Run as root)

1. `uptime; w; top -b -n 1`
   → System load and activity snapshot.
2. `ps auxww --sort=-%cpu | head -n 20`
   → Top running processes.
3. `ss -tulpn`
   → Active network sockets and listening ports.
4. `sudo journalctl -u application --since "1 hour ago"`
   → Recent application logs.
5. `sudo dmesg -T | tail -n 50`
   → Recent kernel messages and errors.

Senior Insight

- A restart can fix the symptom but destroys the evidence.
- Always capture a minimal, focused set of data before recovery.

Outcome

- Preserve the live system state for analysis.
- Identify the real root cause (e.g., resource leak, hung process, or network issue).
- Recover the service with confidence and proof.

VERIQTA

CPU Is 10%, but Every Request Takes 30 Seconds.

Investigate blocked threads, run queues, I/O wait, and application queueing.

The Scenario

- The server looks healthy (CPU ≈10%), but requests are very slow (≈30 seconds).
- No obvious errors, and the application is still running.
- Users are experiencing timeouts and delays.

Where the Delay Can Hide

- **Blocked Threads** (e.g., waiting on DB, lock, or external call)
- **Run Queues** (threads waiting for CPU)
- **I/O Wait** (slow disk, network storage, NFS/EBS)
- **Application Queueing** (requests backing up in the app)

Investigation Steps

1. Check baseline metrics (CPU, load, I/O, and latency).
2. Look for blocked or waiting threads.
3. Check run queue and system pressure.
4. Investigate I/O wait and disk/storage latency.
5. Inspect the application queue and external dependencies (e.g., DB, API).

Key Commands and What to Look For

1. `top or htop`
   → CPU usage, load average, and running processes.
2. `vmstat 1 5`
   → Run queue (r) and I/O wait (wa) should be checked.
3. `iostat -xz 1`
   → Disk I/O latency and utilization (high await = slow I/O).
4. `pidstat -t -p <PID> 1`
   → Thread activity and threads in blocked (D) state.
5. `ss -s or ss -tnp`
   → Check for many connections or a backlog.

Common Root Causes

- Threads blocked on database locks or slow queries.
- High I/O wait from a slow disk or network storage.
- Application queue is full (e.g., web server or worker pool).
- External dependency (e.g., another service or API) is slow.

Senior Insight

- Low CPU does not mean the system is healthy.
- Focus on waits, queues, and dependencies not just CPU usage.
- Correlate system metrics with application traces and request latency.

Outcome

- Identify the real bottleneck (e.g., DB lock, I/O, queue, or external service).
- Apply the right fix (e.g., optimize query, increase worker pool, or improve storage I/O).
- Verify by re-testing that request latency returns to an acceptable level.

VERIQTA

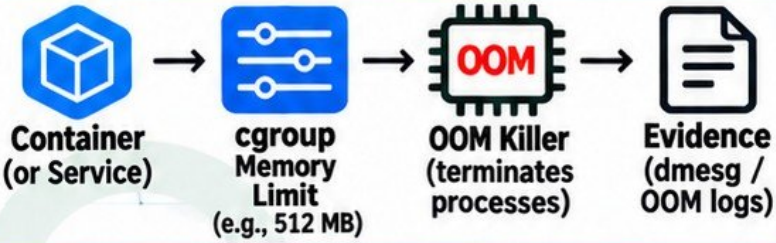
The Server Has Free Memory but Keeps Killing Processes.

Investigate cgroup limits, OOM events, and container memory accounting.

The Scenario

- The server shows free memory (e.g., from free or top), yet processes keep getting killed.
- Logs show "Out of memory: Kill process" (OOM killer).
- The issue is common in containerized or cgroup-limited environments.

What's Really Happening?



Investigation Steps

1. Check overall memory and usage.
2. Verify cgroup/container memory limits.
3. Look for OOM kill events in logs.
4. Identify which process was killed and why.
5. Review application and container memory accounting (RSS, cache, limit).

Key Commands and Expected Evidence

1. `free -h`
   → Shows total, used, and free memory (e.g., free -h).
2. `dmesg | grep -i "oom"`
   → OOM kill event with the killed process and reason.
3. `cat /sys/fs/cgroup/memory.max`
   → cgroup memory limit (e.g., 536870912 = 512 MB).
4. `docker stats or kubectl top pod`
   → Container memory usage and limit (e.g., 480Mi / 512Mi).
5. `ps -eo pid,ppid,cmd,%mem --sort=-%mem | head`
   → Top memory-consuming processes.

Common Root Causes

- The process hits a cgroup/container memory limit (even if the host has free memory).
- OOM killer invoked due to memory pressure.
- Memory spikes, leaks, or unbounded cache.
- Incorrect or too-low container memory limits.

Senior Insight

- "Free memory" on the host does not mean the process is safe.
- The OOM killer enforces cgroup limits, not just total system memory.
- Always analyze OOM logs and container limits together before taking action.

Outcome

- Confirm whether the kill was due to a cgroup limit or real memory exhaustion.
- Right-size the memory limit or fix the memory leak/usage pattern.
- Validate the fix by monitoring memory, OOM events, and process stability.

VERIQTA

Disk Space Is Available, but Writes Fail.

Find inode exhaustion, deleted-open files, read-only remounts, and filesystem errors.

The Scenario

- df shows plenty of free space, but writes fail.
- Applications return "Read-only file system" or "No space left on device".
- Logs hint at inode or filesystem issues.

Common Causes

- **Inode Exhaustion** (no more inodes)
- **Deleted-Open Files** (files still held)
- **Read-Only Remount** (fs set to ro)
- **Filesystem Errors** (corruption, I/O errors)

Investigation Steps

1. Check disk usage and inode stats: df -h, df -i.
2. Find deleted-open files: lsof | grep deleted.
3. Check filesystem state: mount, dmesg, journal logs.
4. Look for errors: dmesg | grep -i error.
5. Inspect filesystem type and health: fsck -n /dev/sdX.

Key Commands

1. `df -h` → Check disk space.
2. `df -i` → Check inode usage.
3. `lsof | grep deleted` → Find deleted files.
4. `mount` → Check mount options (ro/rw).
5. `dmesg | tail -50` → Look for filesystem errors.

Common Fixes

- Clean up old files and logs.
- Release deleted files (restart if needed).
- Remount filesystem read-write: mount -o remount,rw /mount/point.
- Run fsck if errors persist.

Pro Tips

- Monitor inode usage, not just disk space.
- Check for long-running processes holding deleted files.
- Use log rotation and proper cleanup.
- Investigate hardware if errors repeat.

VERIQTA

The Network Is Reachable, but Connections Keep Dying.

Investigate TCP retransmissions, backlog overflow, conntrack, and ephemeral ports.

The Scenario

- You can ping the server and it looks reachable.
- But client connections often time out or reset.
- The issue is intermittent and worsens under load.

Key Areas to Investigate

- **TCP Retransmissions** (lost or repeated packets)
- **Backlog Overflow** (connection queue is full)
- **Conntrack** (connection tracking table limits)
- **Ephemeral Ports** (out of available ports)

Investigation Steps

1. Verify the problem and scope (reproduce under load).
2. Check TCP retransmissions, errors, and resets.
3. Inspect the listen backlog (queue) on the server.
4. Review conntrack table usage and limits.
5. Check ephemeral port range and port exhaustion.

Key Commands

1. `ss -s`
   → Summary of TCP connections (retrans, resets, errors).
2. `netstat -s | grep -i retrans`
   → View TCP retransmission statistics.
3. `ss -ltn`
   → Check listening sockets and backlog (Recv-Q).
4. `conntrack -S`
   → Check conntrack usage and drops.
5. `cat /proc/sys/net/ipv4/ip_local_port_range`
   → View ephemeral port range (e.g., 32768 60999).

Common Root Causes

- High TCP retransmissions due to packet loss or network congestion.
- Listen backlog overflow (too many incoming connections).
- Conntrack table full (new connections dropped).
- Ephemeral port exhaustion on the client or server.

Outcome

- Identify the exact bottleneck (retransmissions, backlog, conntrack, or ports).
- Apply the right fix (tune backlog, increase conntrack, expand port range, or reduce load).
- Validate with repeat tests to confirm stable connections.

DNS Works on the Host but Fails Inside the Application.

Investigate resolver differences, negative caching, and namespace isolation.

The Scenario

- On the server, DNS works (e.g., nslookup or dig succeeds).
- Inside the application (container or service), DNS lookup fails.
- The app cannot resolve a domain (e.g., api.veriqta.com).

Architecture at a Glance



Investigation Steps

1. Confirm the problem (host vs. application).
2. Check DNS configuration (/etc/resolv.conf).
3. Test DNS from inside the application/container.
4. Check for negative caching (e.g., nscd, systemd-resolved).
5. Look for namespace isolation (e.g., container or network policies).

Key Commands and Checks

1. `dig api.veriqta.com`
   → Test DNS on the host (should succeed).
2. `docker exec -it app cat /etc/resolv.conf`
   → Check the DNS resolver inside the container.
3. `docker exec -it app dig api.veriqta.com`
   → Test DNS inside the application (likely fails).
4. `resolvectl status | grep -i dns`
   → Check system DNS and caching.
5. `ip netns list` or `docker network inspect <name>`
   → Verify namespace/network isolation and DNS settings.

Common Root Causes

- Different DNS resolver inside the container.
- Negative DNS caching (old failed result still cached).
- Network namespace isolation (container uses its own DNS).
- Missing or misconfigured DNS in the container/network policy.

Outcome

- Identify why the app cannot resolve DNS even when the host can.
- Fix the resolver, clear negative cache, or adjust container/network settings.
- Verify with a DNS test from both the host and the application.

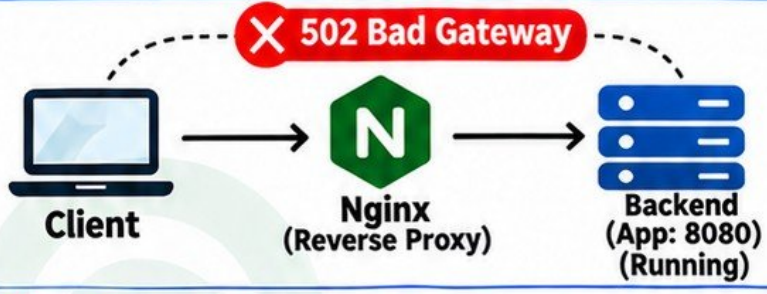
Nginx Returns 502, but the Backend Is Running.

Trace upstream sockets, keepalive behavior, worker limits, and timeout mismatches.

The Scenario

- Users get "502 Bad Gateway" from Nginx.
- The backend service is running and healthy (e.g., on port 8080).
- Nginx cannot successfully proxy requests to the backend.

Request Flow and Failure Point



Investigation Steps

1. Confirm the backend is up and listening (health and port).
2. Check Nginx error logs for the exact 502 reason.
3. Trace the upstream socket connection from Nginx.
4. Review keepalive, worker limits, and connection settings.
5. Compare Nginx and backend timeouts for mismatches.

Key Commands

1. `systemctl status nginx backend`
   → Verify both services are running.
2. `ss -tna | grep 8080`
   → Confirm the backend is listening.
3. `tail -n 50 /var/log/nginx/error.log`
   → See the exact upstream error (e.g., timeout or refused).
4. `curl -v http://127.0.0.1:8080/health`
   → Test the backend directly from the Nginx host.
5. `nginx -T | grep -E 'upstream|keepalive|timeout|worker'`
   → Review upstream, keepalive, worker, and timeout settings.

Common Root Causes

- Backend is slow or hit by timeout (proxy_read_timeout / proxy_connect_timeout).
- Keepalive connection issues (stale or closed).
- Nginx worker or connection limit reached (worker_connections / worker_processes).
- Upstream socket errors (connection reset, refused, or no available upstream).

Outcome

- Identify the exact reason for the 502 (timeout, connection, limit, or keepalive).
- Apply the right fix (adjust timeouts, keepalive, workers, or upstream config).
- Verify with end-to-end test (client → Nginx → backend) that requests succeed (HTTP 200).

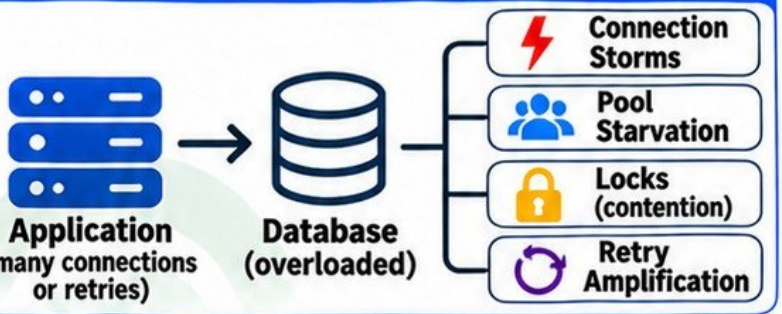
The Database Is Healthy Until the Application Connects.

Diagnose connection storms, pool starvation, locks, and retry amplification.

The Scenario

- The database looks healthy (e.g., low CPU, normal metrics).
- As soon as the application connects, performance degrades (slow, timeouts, or errors).
- The issue is often caused by excessive connections, pool exhaustion, locks, or aggressive retries.

What's Happening



Investigation Steps

1. Check current connections (count, source, and state).
2. Inspect the application's connection pool settings (size, idle, timeout).
3. Look for blocking locks and long-running queries.
4. Review retry and backoff settings (e.g., rapid retries causing more load).
5. Correlate database metrics before and after the application connects.

Key Commands

1. `(PostgreSQL) psql -c "select count(*) from pg_stat_activity;"`
2. `(MySQL) SHOW PROCESSLIST;`
3. `(PostgreSQL) psql -c "select * from pg_locks;"`
4. `tail -n 100 /var/log/app.log | grep -i retry`
5. `ss -tnp | grep 5432 (or your DB port)`

Common Root Causes

- Connection storm (sudden spike in connections).
- Connection pool starvation (all pool connections in use).
- Blocking locks (sessions waiting on the same data).
- Retry amplification (rapid, repeated retries increasing load).

Outcome

- Identify the exact cause (connections, pool, lock, or retry).
- Apply the right fix (tune pool size, release locks, limit retries, or throttle connections).
- Restore stable and predictable database performance.

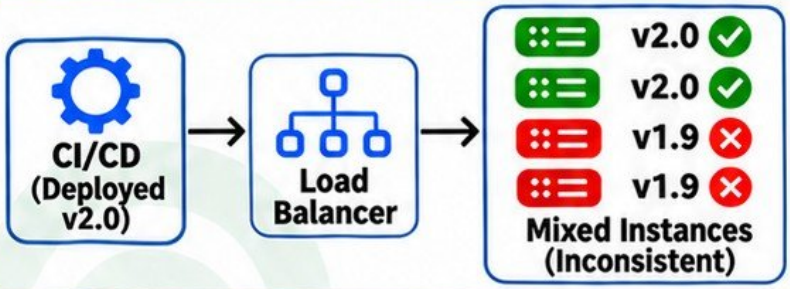
The Deployment Passed, but Only Some Users Are Failing.

Investigate stale instances, inconsistent configurations, and partial rollouts.

The Scenario

- The deployment is marked successful.
- Most users are fine, but a subset are experiencing errors (e.g., 500, feature missing, or inconsistent behavior).
- The issue is environment or instance-specific, not a full outage.

Deployment Flow (Partial Rollout)



Investigation Steps

1. Verify deployment status and version across all instances.
2. Check load balancer targets (healthy vs. unhealthy, version).
3. Compare configuration, environment variables, and feature flags.
4. Identify and route traffic to the correct (updated) instances.
5. Fix the drift and complete a consistent rollout.

Key Commands

1. `kubectl get pods -o wide`
   → Check pod versions (Kubernetes).
2. `kubectl rollout status deploy/<app>`
   → Verify rollout progress and status.
3. `kubectl get pods -l app=<app> -o jsonpath='{.items[*].spec.containers[*].image}'`
   → Confirm image version across pods.
4. `kubectl describe pod <pod> | grep -i version`
   → Check runtime version/config on a specific pod.
5. `kubectl rollout restart deploy/<app>`
   → Complete a consistent rollout to all instances.

Common Root Causes

- Partial rollout (some instances still running the old version).
- Configuration drift (different env vars, secrets, or feature flags).
- Stale instances not refreshed/terminated.
- Load balancer still routing traffic to old or misconfigured instances.

Outcome

- Identify which users are hitting the old or misconfigured instances.
- Align versions and configurations across all instances.
- Rerun rollout and verify a consistent, stable experience for all users.

The Container Is Healthy but Cannot Serve Traffic.

Examine misleading probes, CPU throttling, PID limits, and network namespaces.

The Scenario

- The container shows "Healthy" (e.g., Ready/Liveness = OK).
- However, it cannot serve requests (timeouts or connection failures).
- The health check gives a false sense of availability.

Diagnostic Focus

- **Probes** (false OK?)
- **CPU Throttling** (resource limits)
- **PID Limits** (process exhaustion)
- **Network Namespaces** (isolation/ connectivity)

Investigation Steps

1. Review the liveness/readiness probe configuration.
2. Check for CPU throttling and resource limits (cgroups).
3. Verify PID limit and current process count.
4. Confirm network namespace and connectivity from inside the container.
5. Correlate container metrics, logs, and application behavior.

Key Commands

1. `kubectl describe pod <pod>`
   → View probe config and recent events.
2. `kubectl top pod <pod>`
   → Check CPU usage and throttling.
3. `cat /sys/fs/cgroup/cpu.max`
   → See CPU limits (cgroups v2).
4. `cat /sys/fs/cgroup/pids.max`
   → Check PID limit and current usage.
5. `kubectl exec -it <pod> -- ss -tulnp`
   → Verify the app is listening on the expected port.

Common Root Causes

- Misleading health probes (check passes but the app is not ready).
- CPU throttling (container is starved of CPU).
- PID limit reached (cannot start new processes).
- Network namespace issues (cannot bind, route, or reach the service).

Outcome

- Identify why the container is "healthy" but not serving traffic.
- Correct the probe, resource limits (CPU/PID), or network configuration.
- Validate with a real traffic test to confirm the container is fully available.

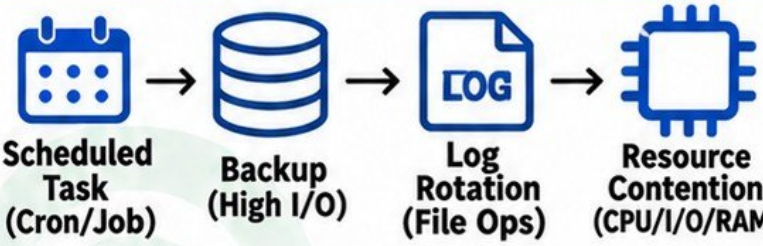
The Server Fails at Exactly the Same Time Every Day.

Correlate cron jobs, backups, log rotation, and resource contention.

The Scenario

- The server becomes slow, unresponsive, or fails at the same time daily (e.g., 2:00 a.m.).
- It recovers later without manual intervention.
- The pattern suggests a scheduled task or resource-heavy job.

Daily Failure Pattern



Investigation Steps

1. Confirm the exact failure time and duration.
2. Review scheduled jobs (cron, systemd timers).
3. Check backup and log rotation jobs (size, runtime, and impact).
4. Monitor resource usage at the failure time (CPU, memory, disk I/O).
5. Test by pausing or rescheduling the suspected job to verify.

Key Commands

1. `crontab -l`
   → List user cron jobs.
2. `sudo crontab -l`
   → List system cron jobs.
3. `sudo systemctl list-timers`
   → Show all scheduled system timers.
4. `journalctl -u <service> --since "today"`
   → Check logs around the failure time.
5. `top or iostat -x 1`
   → Monitor CPU, memory, and disk I/O during the event.

Common Root Causes

- CPU or disk I/O spikes from backups (e.g., large database dumps).
- Log rotation compressing large log files (high disk and CPU usage).
- Multiple scheduled jobs running at the same time (resource contention).
- Cron job loops or long-running tasks that overlap with other services.

Outcome

- Identify the scheduled task(s) causing the daily failure.
- Optimize or reschedule the job (stagger timing, reduce load, or limit resources).
- Ensure backups, log rotation, and other jobs run without disrupting the server.

TLS Is Valid, but Clients Still Cannot Connect.

TLS | **Diagnose certificate chains, SNI, trust stores, and protocol negotiation.**

The Scenario

- The TLS certificate is valid (e.g., not expired and correctly installed).
- However, some clients still cannot connect (handshake fails or connection is reset).
- The issue is often in the TLS setup, not the certificate itself.

TLS Connection Flow



Investigation Steps

1. Verify the full certificate chain (server + intermediate CAs).
2. Check for SNI issues (correct hostname and cert).
3. Validate the client's trust store (ensure the CA is trusted).
4. Check the TLS protocol and cipher compatibility (e.g., TLS 1.2/1.3, allowed ciphers).
5. Test the handshake from both working and failing clients.

Key Commands

1. `openssl s_client -connect example.com:443 \\ -servername example.com`
   → Check certificate chain, SNI, and handshake.
2. `openssl s_client -showcerts -connect example.com:443`
   → View the full certificate chain.
3. `curl -vk https://example.com`
   → See detailed TLS negotiation and errors.
4. `nmap --script ssl-enum-ciphers -p 443 example.com`
   → Check supported TLS versions and ciphers.
5. `openssl version -a`
   → Confirm the OpenSSL version and supported protocols.

Common Root Causes

- Incomplete certificate chain (missing intermediate CA).
- SNI mismatch (wrong hostname or certificate).
- Client trust store not updated (doesn't trust the CA).
- Protocol or cipher mismatch (e.g., client only supports TLS 1.2).

Outcome

- Identify the exact reason for the TLS connection failure.
- Fix the certificate chain, SNI, trust store, or protocol settings.
- Verify a successful TLS handshake and client connection.

The Firewall Is Open, but Packets Never Arrive.

Investigate asymmetric routing, security rules, packet captures, and MTU issues.

The Scenario

- The firewall allows the traffic (port/ICMP is open).
- But packets never reach the destination (no response, no logs).
- The issue is often in routing, intermediate network rules, or packet size (MTU).

Traffic Path (with Possible Breaks)



Investigation Steps

1. Confirm the firewall rules and that the port is open.
2. Check for asymmetric routing (different path for return traffic).
3. Capture traffic on both ends (e.g., tcpdump) to see if packets arrive.
4. Check MTU and look for fragmentation or drops.
5. Verify security rules on all intermediate devices (router, SG, NACL, etc.).

Key Commands

1. `sudo iptables -L -n | grep <port>`
   → Verify the firewall allows the traffic.
2. `traceroute <destination-ip>`
   → Check the routing path and for asymmetry.
3. `sudo tcpdump -ni eth0 host <ip> and port <port>`
   → Capture packets and confirm if they arrive.
4. `ping -M do -s 1472 <destination-ip>`
   → Test MTU (find largest working packet size).
5. `ip route show`
   → Review routing entries and possible asymmetry.

Common Root Causes

- Asymmetric routing (return traffic takes a different path and is dropped).
- Security rules elsewhere (router, SG, NACL) blocking the traffic.
- MTU issues (packets too large, dropped or not fragmented).
- Misconfigured routes (wrong next hop or missing route).

Outcome

- Determine why packets are not reaching the server.
- Fix the routing, security rule, or MTU configuration.
- Confirm end-to-end connectivity with ping, traceroute, and packet capture.

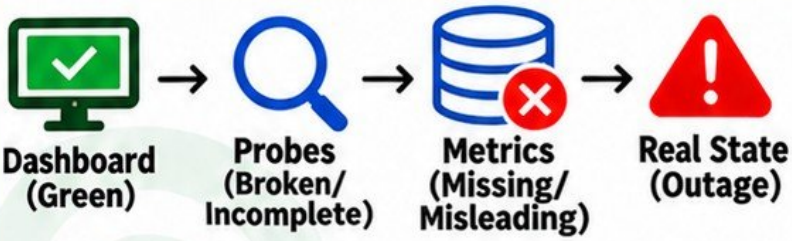
The Monitoring Dashboard Is Green During an Outage.

Reconstruct missing signals, broken probes, and misleading aggregate metrics.

The Scenario

- There is a real outage (users cannot access the service).
- However, the monitoring dashboard still shows green (healthy).
- The issue is usually missing data, broken probes, or metrics that hide the real problem.

Monitoring Visibility Gap



Investigation Steps

1. Confirm the outage with real user tests (e.g., curl, ping, or a synthetic check).
2. Validate the health probes (endpoint, path, and response).
3. Check metrics ingestion (are metrics being collected and updated?).
4. Review dashboard queries (aggregation, time range, filters).
5. Correlate with logs, traces, and downstream dependency checks.

Key Commands

1. `curl -I https://<service-url>`
   → Test the service from the outside.
2. `kubectl get pods -A`
   → Check the actual state of the service.
3. `curl http://<pod-ip>:<port>/health`
   → Verify the probe endpoint directly.
4. `promtool query instant 'up{job="app"}'`
   → Check if metrics are being scraped.
5. `grafana-cli ds ls`
   → Confirm dashboard data sources are reachable.

Common Root Causes

- Broken or misconfigured health probes (returns healthy even when the app is down).
- Missing metrics (scrape failures, agent down, or bad configuration).
- Misleading aggregates (e.g., average hides failing instances).
- Incorrect dashboard filters or time range (showing outdated or partial data).

Outcome

- Rebuild reliable probes that reflect real service health.
- Restore and validate all required metrics collection.
- Use per-instance views, proper thresholds, and no-to-end alerts to prevent false green.

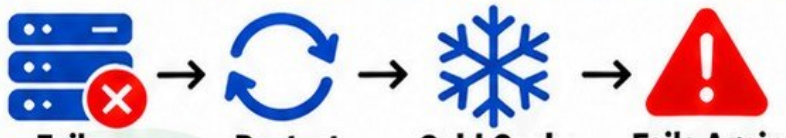
The Server Recovers, Then Immediately Fails Again.

Investigate retry storms, cold caches, startup dependencies, and cascading overload.

The Scenario

- The server goes down, recovers after a restart, but soon fails again.
- The failure repeats in a short cycle (e.g., every few minutes).
- The root cause is often retries from clients, cold caches, or dependent services not being ready.

Failure Cycle



Investigation Steps

1. Review the timeline and logs (across app, cache, and dependencies).
2. Check for retry storms (multiple clients retrying at once).
3. Verify cache behavior after restart (cold cache or cache miss).
4. Confirm startup dependencies (DB, cache, message queue, etc.) are ready before the app starts.
5. Trace the load and resource usage during recovery (CPU, memory, I/O).

Key Commands

1. `grep -i "retry\|timeout" /var/log/app.log`
   → Check for retries and timeouts.
2. `redis-cli info stats`
   → Review cache hits/misses and load.
3. `systemctl status <dependency>`
   → Verify required services are healthy (DB, MQ, etc.).
4. `top -o %CPU | head -n 10`
   → Check resource spikes after recovery.
5. `journalctl -u <app.service> --since "1 hour ago"`
   → Trace the fail → recover → fail cycle.

Common Root Causes

- Retry storms (many clients retry at once, overloading the server).
- Cold caches (cache is empty after restart, causing high load).
- Startup dependencies not ready (e.g., DB, cache, or queue still starting).
- Cascading overload (one slow service triggers failures in others).

Outcome

- Identify the trigger that causes the repeated failure.
- Stabilize restarts (add backoff, jitter, and retry limits).
- Warm the cache and ensure dependencies are ready before the app goes live.
- Prevent cascading overload with proper limits and circuit breakers.

The Fix Worked in Staging but Broke Production.

Trace configuration drift, environment differences, and hidden dependencies.

The Scenario

- A change is tested and works in staging.
- The same change fails in production (errors, degraded performance, or downtime).
- The cause is usually differences in configuration, versions, resources, or hidden dependencies.

Environment Comparison



Investigation Steps

1. Compare configurations (ENV vars, features, flags, limits).
2. Check version differences (app, DB, libraries, runtime).
3. Verify infrastructure and resources (CPU, memory, storage, network, quotas).
4. Identify hidden dependencies (third-party services, data, or specific settings).
5. Reproduce production conditions in staging (env parity) and test.

Key Commands

1. `diff -u staging.env production.env`
   → Compare environment variables.
2. `kubectl get configmap -A`
   → Check config differences (Kubernetes).
3. `docker inspect <container>`
   → Review runtime settings and image versions.
4. `npm ls --depth=0 | grep <package>`
   → Verify dependency versions.
5. `terraform plan -var-file=prod.tfvars`
   → Compare infrastructure configuration.

Common Root Causes

- Configuration drift (different settings between staging and production).
- Version mismatch (app, library, DB, or runtime).
- Environment differences (data, limits, network, or external services).
- Hidden or untested dependencies (e.g., API, license, or feature flag).

Outcome

- Find the exact difference causing the production failure.
- Align configurations, versions, and dependencies across environments.
- Ensure environment parity and add checks to prevent future drift.

The Incident Looks Like an Attack, but Is It?

Distinguish legitimate traffic spikes, resource abuse, and malicious activity using evidence.

The Scenario

- The system shows a sudden spike in traffic and high resource usage.
- It looks like an attack (e.g., DDoS or abuse).
- However, it could also be legitimate traffic (e.g., a product launch, campaign, or viral event).

Traffic Classification



Investigation Steps

1. Review traffic patterns (volume, source IPs, and regions).
2. Check logs and request behavior (rate, endpoints, error codes).
3. Compare with known business events (release, campaign, or external traffic).
4. Look for signs of abuse (unusual rates, repeated requests, or known bad IPs).
5. Correlate with resource metrics (CPU, memory, bandwidth, and throttling).

Key Commands

1. `tail -f /var/log/nginx/access.log`
   → Inspect incoming requests in real time.
2. `awk '{print $1}' access.log | sort | uniq -c | sort -nr`
   → Find top source IPs (highest traffic).
3. `sudo netstat -antp | grep ESTABLISHED`
   → Check active connections and usage.
4. `top or htop`
   → Monitor CPU and memory for resource abuse.
5. `sudo tcpdump -i eth0 -n "host <suspicious-ip>"`
   → Capture and inspet traffic from a suspect IP.

Common Root Causes

- Legitimate traffic surge (product launch, marketing, or viral traffic).
- Resource abuse (high request rate, expensive endpoints, or scraping).
- Malicious activity (DDoS, brute force, or bot traffic).
- Lack of rate limiting or traffic controls causing overload.

Outcome

- Confirm whether the incident is legitimate, abusive, or malicious.
- Apply the right response (scale, rate-limit, block, or mitigate).
- Document the evidence and update monitoring and protection rules.

The 3 A.M. Incident: Five Symptoms, One Hidden Cause.

Full investigation combining Nginx, Linux, database, DNS, and deployment evidence.

The Scenario

- At 3:00 A.M., the system degrades with multiple issues.
- The symptoms look unrelated across different layers.
- A single hidden cause is behind all five symptoms.

The Five Symptoms (Observed)

1. 502/504 Errors (Nginx)
2. High CPU/Load (Linux)
3. Slow Queries (Database)
4. Intermittent DNS Resolution
5. Failed/ Stuck Deployment

Investigation Steps

1. Check Nginx (error logs, upstream, connection timeouts).
2. Analyze Linux (CPU, memory, I/O, processes, and system logs).
3. Review database (slow queries, connections, and locks).
4. Verify DNS (record resolution, TTL, and latency).
5. Correlate with deployment (latest changes, rollouts, and timing at 3 A.M.).

Key Commands

1. `tail -n 100 /var/log/nginx/error.log`
   → Check Nginx errors at 3 A.M.
2. `top or htop`
   → Inspect CPU, memory, and process usage.
3. `journalctl -u nginx --since "3:00"`
   → Review Nginx service logs.
4. `mysql -e "SHOW PROCESSLIST;"`
   → Identify slow or blocked database queries.
5. `dig +trace yourdomain.com`
   → Verify DNS resolution and response path.

The Hidden Cause (Example)

- A scheduled job at 3 A.M. (e.g., backup, data sync, or report) exhausted resources (CPU/I/O).
- This caused slow database response, Nginx upstream timeouts, DNS delays (from high load), and a failed deployment (health checks timing out).

Outcome

- Identified one root cause for all five symptoms.
- Removed or rescheduled the 3 A.M. job.
- Stabilized Nginx, database, DNS, and deployments.
- Added monitoring and alerts to prevent recurrence.

Final Senior SRE Challenge: Recover a Broken Production Platform.

A multi-failure capstone with incident command, evidence, recovery, verification, and postmortem.

The Scenario

- The production platform is partially or fully down.
- Multiple issues occur at the same time across the stack.
- You lead the end-to-end response from diagnosis to postmortem.

Multi-Failure Environment



Your SRE Tasks

1. Activate incident command (roles, comms, and timeline).
2. Collect and correlate evidence (logs, metrics, traces, events).
3. Identify and fix root causes (across app, DB, network, and infra).
4. Restore and verify service health (end-to-end checks and SLIs/SLOs).
5. Conduct a postmortem with lessons learned and prevention actions.

Key Commands

1. `kubectl get pods -A`
   → Check cluster and workload health.
2. `curl -I https://yourapp.com`
   → Verify service availability and response.
3. `top / htop`
   → Inspect CPU, memory, and running processes.
4. `journalctl -u nginx --since "1 hour ago"`
   → Review web server logs for errors.
5. `dig yourapp.com +short`
   → Validate DNS resolution and routing.

Possible Failure Areas

- Web tier (Nginx): 502/504 errors or upstream failures.
- Database: high load, slow queries, or connection limits.
- Kubernetes: crashed pods, resource pressure, or unhealthy deployments.
- DNS/Network: resolution delays or routing issues.
- Monitoring: missing or delayed alerts (visibility gap).

Outcome

- Platform fully restored and verified.
- Clear incident timeline and root cause(s).
- Documented postmortem with corrective and preventive actions.
- Improved resilience (better alerts, limits, redundancy, and runbooks).